

Programación Funcional

Ejercicios de Práctica Nro.2

Sistemas de tipos

Aclaraciones:

- Los ejercicios siguen un orden de complejidad creciente, y cada uno puede servir a los siguientes. No se recomienda saltar ejercicios sin consultar antes a un docente.
- Recordar que se pueden aprovechar en todo momento las funciones ya definidas, tanto las de esta misma práctica como las de prácticas anteriores.
- Probar todas las implementaciones, al menos en una consola interactiva.
- Es sumamente aconsejable resolver los ejercicios utilizando primordialmente los conceptos y metodologías vistos en clase, dado que los exámenes de la materia evalúan principalmente este aspecto. Para utilizando formas alternativas al resolver los ejercicios consultar a los docentes.

Ejercicio 1) Indicar los tipos de las siguientes definiciones:

- | | |
|--|---|
| a. <code>first (x,y) = x</code> | <code>first :: (a, b) -> a</code> |
| b. <code>apply f = g</code>
<code>where g x = f x</code> | <code>apply :: (a -> b) -> (a -> b)</code> |
| c. <code>twice f = g</code>
<code>where g x = f (f x)</code> | <code>twice :: (a -> a) -> (a -> a)</code> |
| d. <code>doble x = x + x</code> | <code>doble :: Int -> Int</code> |
| e. <code>swap (x, y) = (y, x)</code> | <code>swap :: (a, b) -> (b, a)</code> |
| f. <code>uflip f = g</code>
<code>where g p = f (swap p)</code> | <code>uflip :: ((a, b) -> c) -> ((b, a) -> c)</code> |

Ejercicio 2) Dadas las definiciones anteriores, indicar el tipo de las siguientes expresiones:

- | | |
|-------------------------------------|---|
| a. <code>apply first</code> | <code>apply first :: (a, b)</code> |
| b. <code>first (swap, uflip)</code> | <code>first (swap, uflip) :: (a, b) -> (b, a)</code> |
| c. <code>twice doble</code> | <code>twice doble :: Int -> Int</code> |
| d. <code>twice twice</code> | <code>twice twice :: (a -> a) -> (a -> a)</code> |
| e. <code>twice uflip</code> | <code>twice uflip :: ((a, a) -> c) -> ((a, a) -> c)</code> |
| f. <code>twice swap</code> | <code>twice swap :: (a, a) -> (a, a)</code> |
| g. <code>uflip swap</code> | <code>uflip swap :: (b, a) -> (b, a)</code> |
| h. <code>(twice twice) swap</code> | <code>(twice twice) swap :: (a, a) -> (a, a)</code> |

Ejercicio 3) Dadas las siguientes definiciones y los siguientes tipos, asociar cada tipo con la función correspondiente.

- | | |
|---|---|
| a. <code>const x = g</code>
<code>where g y = x</code> | <code>a -> (b -> a)</code> |
| b. <code>appDup f = g</code>
<code>where g x = f (x, x)</code> | <code>((a, a) -> b) -> (a -> b)</code> |

- c. `appFork (f, g) = h` (a -> b, a -> c) -> (a -> (b, c))
`where h x = (f x, g x)`
- d. `appPar (f, g) = h`
`where h (x, y) = (f x, g y)` (a -> b, c -> d) -> ((a, c) -> (b, d))
- e. `appDist f = g` (a -> b) -> ((a, a) -> (b, b))
`where g (x, y) = (f x, f y)`
- f. `flip f = h`
`where h x = k` (a -> (b -> c)) -> (b -> (a -> c))
`where k y = (f y) x`
- g. `subst f = h`
`where h g = k` (a -> (b -> c)) -> ((a -> b) -> (a -> c))
`where k x = (f x) (g x)`
- I. ✗ `(a -> b, c -> d) -> ((a, c) -> (b, d))`
- II. `((a, a) -> b) -> (a -> b)`
- III. `(a -> (b -> c)) -> (b -> (a -> c))`
- IV. `(a -> b) -> ((a, a) -> (b, b))`
- V. `(a -> b, a -> c) -> (a -> (b, c))`
- VI. `(a -> (b -> c)) -> ((a -> b) -> (a -> c))`
- VII. `a -> (b -> a)`

Ejercicio 4) Para cada una de las siguientes expresiones decidir si poseen tipo. Si es así indicar cuál es.

- a. `1 && 2 == 2` No tipa
- b. `1 + if 3 < 5 then 3 else 5` Int
- c. `let par = (True, 4)`
`in (if first par then first par else second par)` Raro, pero es un bool. Según ghci no tipa
- d. `(doble doble) 5` No tipa
- e. `doble (doble 5)` Int
- f. `twice first` No tipa
- g. `(twice doble) doble` Int -> Int
- h. `(twice twice) first` No tipa
- i. `apply apply` (a -> b) -> (a -> b)

Ejercicio 5) Dar dos ejemplos de expresiones que tengan cada uno de los siguientes tipos:

- a. `Bool` True, 4>5
- b. `(Int, Int)` (doble 2, 5) | (7, twice (doble 3))
- c. `Char -> Int` \x -> 4 | \x -> if x == 'c' then 1 else 0
- d. `(Int, Char) -> Bool` \x, c -> if c == 'a' then x else x-1 | first
- e. `(Int -> Int) -> Int` \x, y -> x + y | second
- f. `(Bool -> Bool, Int)` (not, 3) | ((True &&, 4)
- g. `a -> Bool` \x -> True | \x -> False

Ejercicio 6) Para cada una de las siguientes expresiones, decir a cuál función del ejercicio 3 es equivalente. Ofrecer argumentos de por qué son equivalentes.

- a. `\p -> let (f, g) = p
 in \x -> (f x, g x)` *appFork*
- b. `\f -> (\g -> (\x -> f x (g x)))` *subst*
- c. `\f -> (\x -> (\y -> (f y) x))` *flip*
- d. `\f -> (\px -> let (x, y) = px
 in (f x, f y))` *appDist*
- e. `\x -> (\y -> x)` *const*
- f. `\pf -> let (f, g) = pf
 in \px -> let (x, y) = px
 in (f x, g y)` *appPar*
- g. `\f -> (\x -> f (x, x))` *appDup*

Ejercicio 7) Encontrar cuales de estas expresiones son equivalentes entre sí.

Sugerencia: utilizar funciones anónimas es una forma interesante de encontrar equivalencias entre expresiones que denotan funciones.

- a. `appFork id,id`
- b. `\f -> appDup (appDist f)`
- c. `appDup id`
- d. `appDup appFork`
- e. `flip (appDup const)`
- f. `const (appDup id)`